# Concepts Shared by All Predictors: Colloquium Textbook

**Thesis:** A Unified Benchmark of Grokking Predictors in Neural Networks · Common concepts (not predictor-specific) · Prepared 5 November 2026

This is the study book for the concepts shared by every predictor in your benchmark: 7 modules and 29 topics, in the order you'd meet them, from grokking itself to research honesty. It doesn't describe any single predictor; those come later, in their own books. Every event list used in a worked example is made up for illustration and belongs to no predictor of yours. Every term is defined in the Notation and glossary chapter before it's used.

Jonathan John · M.Sc. AI

# Contents

**Notation and glossary** **3**

**1 The grokking phenomenon** **9**

1.1 Grokking . . . . . 9

1.2 Memorisation vs generalisation . . . . . 9

1.3 Circuit formation and cleanup . . . . . 10

**2 Your training setup** **11**

2.1 Modular addition task . . . . . 11

2.2 Train/test split (30%) . . . . . 12

2.3 Transformer, logits and cross-entropy loss . . . . . 13

2.4 AdamW and weight decay . . . . . 15

2.5 Epochs and checkpoints . . . . . 16

2.6 Seeds and why five . . . . . 17

**3 Measuring the outcome** **19**

3.1 Test accuracy . . . . . 19

3.2 The grok epoch $G_i$ . . . . . 20

**4 What a predictor is** **21**

4.1 Predictor and event epoch $P_i$ . . . . . 21

4.2 Lead and the 100-epoch margin . . . . . 21

4.3 Lead fraction . . . . . 22

4.4 Relative lead and mean relative lead . . . . . 23

4.5 Misses and `None` . . . . . 25

**5 Spearman's $\rho$ and small samples** **27**

5.1 Spearman rank correlation $\rho$ . . . . . 27

5.2 Pearson vs Spearman . . . . . 28

5.3 Only 21 possible values with 5 seeds . . . . . 30

5.4 Significance with small $n$ . . . . . 31

**6 The frozen protocol and the PASS rule** **33**

6.1 The frozen protocol . . . . . 33

6.2 The PASS rule . . . . . 33

6.3 Pre-registration and criterion validation . . . . . 35

**7 Research honesty** **37**

7.1 Hindsight (look-ahead) bias . . . . . 37

7.2 Causal vs centred moving average . . . . . 38

7.3 Researcher degrees of freedom . . . . . 39

7.4 Sensitivity analysis . . . . . 40

7.5 Baselines, controls and noise floor . . . . . 41

7.6 Negative results and reproducibility . . . . . 41

**Recommended reading** **42**

# Notation and glossary

Every symbol and term in this book is defined here, once, in dependency order: each entry uses only entries above it. The modules use only these names. Entries are numbered **A1**, **N1**, **N2**, … so the text can point back to them. For each entry you get what it is, its type, where it's evaluated, its sign or direction where that matters, and its role in the benchmark.

## Part A. How to read the maths

**A1 Set, $\in$, #.** A set is a collection of objects written in braces, e.g. $\{0, 1, 2\}$. $x \in S$ means "$x$ belongs to the set $S$". $\#S$ is an integer: the number of objects in $S$.

**A2** $\min\{\ldots\}$**.** The smallest number in a set of numbers.

**A3** $\lfloor x \rfloor$ **(floor).** For a scalar $x \geq 0$, the largest integer that is $\leq x$. Python's `int()` gives the same result for $x \geq 0$.

**A4 mod.** For whole numbers $x \geq 0$ and $y > 0$, $x$ mod $y$ is the remainder when $x$ is divided by $y$: a whole number from $0$ to $y - 1$. For example, $150$ mod $113 = 37$. ($x$ and $y$ are placeholders in this entry only.)

**A5** $=$ **and** $\approx$**.** $=$ means exactly equal. $\approx$ means approximately equal. In a derivation $\approx$ appears once, on the line where digits or terms are dropped, and that line says what was dropped.

**A6** $\because$ **and** $\therefore$**.** $\because$ means "because"; every line of a solution ends with one. $\therefore$ means "therefore"; it opens the conclusion.

**A7 Derivation layout (H.K. Dass).** *Given*: a numbered list of facts, each naming where it comes from (a glossary entry, a section, or a file of yours). *Let*: each symbol used, defined once. *To find*: what is computed or shown. *Solution*: one aligned block, one step per line. Then a $\therefore$ sentence in plain words and a boxed answer.

**A8 d.p.** Decimal places. "0.991 (3 d.p.)" means the value rounded to 3 decimal places.

**A9 Your files.** `nanda_unified.yaml`: the configuration (config) file of the final setup. `analyze_nanda_unified.py`: the analysis script. `summary.json`: the saved results file. `context.md`: your project log.

**A10 Citations [1]–[5].** Numbers in square brackets point to the Recommended reading list at the end. "Power et al." is [1]; "Nanda et al." or "Nanda" is [2].

**A11 Viva, examiner, attack point, audit finding.** The viva is your oral exam (colloquium). The examiner asks the questions. An attack point is a weakness in your work that an examiner could raise there. An audit finding is an issue found in the 3 November 2026 audit of your code.

## Part B. The task and the data

**N1** $p$**, the modulus.** Integer, prime, $p = 113$. Set in `nanda_unified.yaml` and never changed. Role: sets the size of the task.

**N2** $a$, $b$**, the inputs.** Two integers, each in $\{0, 1, \ldots, p-1\} = \{0, \ldots, 112\}$. Role: the two numbers the model reads.

**N3** $c$**, the correct answer.** Integer, $c = (a + b) \bmod p$, so $c \in \{0, \ldots, 112\}$. Role: the label the model must output.

**N4 Pair and** $N$**.** A pair is one example: the inputs $(a, b)$ together with their answer $c$. $N$ is an integer, the number of all pairs: $N = p^2 = 12{,}769$ (worked out in Section 2.1).

**N5 The model, its weights** $\theta$**, and training.** The model is a one-layer, four-head transformer (a standard type of neural network; "one-layer, four-head" names its size, and its insides aren't needed here). It's a function: it reads a pair's inputs $(a, b)$ and a weight vector $\theta$, and returns 113 scores, one per possible answer. $\theta$ is a vector with $M$ entries $\theta_1, \ldots, \theta_M$: every trainable number of the model. $M$ is a positive integer whose value this book doesn't need. Training means changing $\theta$ again and again so that the model's answers on training pairs get better. Nanda's model also has a weight matrix called $W_O$ (the attention output matrix); yours doesn't. That's a known gap.

**N6** $f$**, the train fraction.** Scalar, $f = 0.3$. It's hard-coded in `get_dataloaders` as `train_size = int(0.3 * len(...))`; the config entry `train_fraction` doesn't change it. Role: the share of pairs used for training.

**N7** $D_{\text{train}}$**,** $D_{\text{test}}$**,** $N_{\text{train}}$**,** $N_{\text{test}}$**.** $D_{\text{train}}$ is the training set: a set of $N_{\text{train}} = 3{,}830$ pairs that are used to change the weights. $D_{\text{test}}$ is the test set: the other $N_{\text{test}} = 8{,}939$ pairs, never used to change the weights. $N_{\text{train}}$ and $N_{\text{test}}$ are integers (worked out in Section 2.2). Role: $D_{\text{test}}$ measures whether the model has learned the rule.

**N8 Seed** $i$ **and run.** A seed is an integer label $i \in \{0, 1, 2, 3, 4\}$ that fixes the random numbers of one run (the initial weights and the data split). A run is one full training of the model with one seed. Role: 5 seeds give 5 independent, repeatable runs.

## Part C. The model and its training

**N9 Epoch** $t$ **and full batch.** An epoch is one full pass over $D_{\text{train}}$. $t$ is an integer epoch counter, $t \in \{0, 1, \ldots, 40{,}000\}$; $t = 0$ means before any training. Every run lasts 40,000 epochs. Your runs are full batch: all 3,830 training pairs are used together each time $\theta$ is changed, so $\theta$ changes once per epoch.

**N10** $\theta_t$**, the weights after epoch** $t$**.** Vector with $M$ entries, $\theta_t = (\theta_{t,1}, \ldots, \theta_{t,M})$: the value of $\theta$ (**N5**) at the end of epoch $t$. $\theta_0$ is the initial weights. Role: everything the model computes at epoch $t$ comes from $\theta_t$.

**N11** $z$**, the logits.** Vector with $p = 113$ entries $z_0, \ldots, z_{112}$, computed by the model for one pair at $\theta_t$. Entry $z_k$ is the score for answer $k \in \{0, \ldots, 112\}$; a larger $z_k$ means the model favours $k$ more.

**N12** $\hat{k}$**, the predicted answer.** Integer in $\{0, \ldots, 112\}$: the $k$ with the largest $z_k$, at $\theta_t$. The model is correct on a pair when $\hat{k} = c$.

**N13** $q_k$**, the softmax probability.** Scalar in $(0, 1)$, computed from $z$ at $\theta_t$: $q_k = e^{z_k} / \sum_{j=0}^{112} e^{z_j}$, where $j$ is a summation index over the 113 answers. The 113 values $q_k$ add up to 1. $q_c$ is the probability the model gives the correct answer. (The

letter $q$ is used so it doesn't clash with the modulus $p$.)

**N14** $\ell$**, the loss of one pair (cross-entropy).** Scalar, $\ell = -\ln q_c$, at $\theta_t$. Sign: $\ell \geq 0$; $\ell = 0$ when $q_c = 1$, and $\ell$ grows as $q_c$ falls. Role: punishes a low probability on the correct answer.

**N15** $L(\theta_t)$**, the training loss.** Scalar: the average of $\ell$ over the 3,830 pairs of $D_{\text{train}}$, at $\theta_t$. Role: the number training tries to make small.

**N16** $g(\theta_t)$**, the gradient.** Vector with $M$ entries; entry $m$ is $\partial L/\partial\theta_m$, the partial derivative (how fast $L$ changes when only $\theta_m$ changes), evaluated at $\theta_t$. Direction: $g(\theta_t)$ points uphill (the direction in which $L$ grows fastest), so a step along $-g(\theta_t)$ goes downhill.

**N17** $\eta$**, the learning rate.** Scalar, $\eta > 0$, the same at every epoch. Role: the step size of each update. Its value is in the config; this book doesn't need it.

**N18 AdamW,** $\beta_1$**,** $\beta_2$**,** $u_t$**.** AdamW is the optimiser: the method that turns gradients into weight updates [3]. It keeps running averages of the gradient and of the squared gradient; $\beta_1 = 0.9$ and $\beta_2 = 0.98$ are scalars that set how much of the past those averages keep. $u_t$ is the AdamW step: a vector with $M$ entries, computed at epoch $t$ from $g(\theta_0), \ldots, g(\theta_t)$ and $\beta_1, \beta_2$. Direction: like $g$, it points roughly uphill, so $-\eta u_t$ is a step downhill.

**N19** $\lambda$**, the weight decay.** Scalar, $\lambda = 1.0$, the same at every epoch. Sign: $\lambda \geq 0$. Role: shrinks every weight a little at each update.

**N20 The weight update, decoupled, confound.** $\theta_{t+1} = \theta_t - \eta\, u_t - \eta\lambda\, \theta_t$. The middle term is the AdamW step downhill. The last term is the decoupled weight decay: decoupled means it's applied directly to $\theta_t$, separately from $u_t$. It pulls every weight towards 0 (Section 2.4). A confound is a second cause that can produce the same effect as the one you're testing; weight decay is one for any quantity computed from the size of the weights.

**N21 Checkpoint.** A saved copy of $\theta_t$ at one epoch $t$. Checkpoints are saved every 50 epochs and evaluated every 100 epochs; evaluating a checkpoint means computing from it the quantities that need saved weights. Role: some quantities are computed afterwards from checkpoints.

## Part D. The outcome

**N22** $\text{acc}_{\text{train}}(t)$**,** $\text{acc}_{\text{test}}(t)$**.** Scalars in $[0, 1]$: the fraction of pairs in $D_{\text{train}}$ (or $D_{\text{test}}$) on which $\hat{k} = c$, using $\theta_t$. Direction: higher is better. Role: $\text{acc}_{\text{test}}$ measures how well the model does on pairs it was never trained on.

**N23 Memorisation and generalisation.** Memorisation: getting the training pairs right by storing them, so $\text{acc}_{\text{train}}(t)$ is high while $\text{acc}_{\text{test}}(t)$ stays low. Generalisation: getting new pairs right because the rule $(a + b) \bmod p$ has been learned, so $\text{acc}_{\text{test}}(t)$ is high too.

**N24 Grokking.** In one run, $\text{acc}_{\text{train}}(t)$ reaches about 1 early, and much later $\text{acc}_{\text{test}}(t)$ suddenly jumps to high values. It is a switch from memorisation to generalisation.

**N25 Circuit and Nanda's three phases.** A circuit is a set of weights in $\theta_t$ that together compute the general rule $(a+b) \bmod p$. Nanda's three phases [2]: memorisation; circuit formation (the circuit is built while $\text{acc}_{\text{test}}$ stays low); cleanup

(the weights used for memorising are removed, and $\text{acc}_{\text{test}}$ jumps).

**N26 Grok threshold and grok epoch $G_i$.** The grok threshold is the scalar 0.9 (the first entry of `grok_thresholds` in the config). $G_i$ is an integer epoch: $G_i = \min\{t : \text{acc}_{\text{test}}(t) > 0.9\}$ in the run of seed $i$. So $\text{acc}_{\text{test}}(t) \leq 0.9$ at every epoch $t < G_i$. Values: $(G_0, G_1, G_2, G_3, G_4) = (12987, 6921, 9510, 11019, 23747)$. Role: the epoch the benchmark tries to warn about in advance. "Grok" on its own ("before grok") means the grok epoch $G_i$ of that run. The $G_i$ are an outcome of training: they don't depend on how any warning is computed.

## Part E. Predictors and events

**N27 Signal $y_t$, $y_{\text{final}}$.** A signal is a scalar computed at epoch $t$ during training, from $\theta_t$ or from the model's outputs. Example: the training loss $L(\theta_t)$ (**N15**) is a signal. $y_{\text{final}}$ is its value at the last epoch, $t = 40{,}000$.

**N28 Moving average, window $w$, causal, centred, ex-ante.** A moving average replaces each point of a signal curve by the average of $w$ neighbouring points; $w$ is a positive integer, the window. A causal moving average at epoch $t$ uses only points at or before $t$. A centred one uses points on both sides of $t$, so it uses later points too. `uniform_filter1d` (a smoothing function available in your code) is centred. An ex-ante trigger is a rule that uses only the past.

**N29 Rule, fire, event epoch $P_i$, miss, `None`.** A rule (event rule) is a fixed procedure that reads a signal curve and names one epoch. If it names an epoch in the run of seed $i$, the rule fires, and that epoch is the event epoch $P_i$: a scalar epoch (it need not be a whole number). If it never names an epoch, that's a miss, and the code stores `None` instead of a number.

**N30 Rule settings and `skip_epochs`.** The fixed numbers a rule uses, such as a window $w$ (**N28**) or a threshold level. `skip_epochs` is one such integer setting: the number of first epochs a rule ignores. Changing any setting changes the rule.

**N31 Predictor.** A (signal, rule) pair. If the rule or one of its settings changes, it's a different predictor. Your benchmark has eight predictors. This book names none of them: everything in it applies to all eight.

**N32 Illustrative event list.** A made-up list of event epochs $(P_0, \ldots, P_4)$, one per seed, written down only to show how a calculation works. It isn't the output of any of your predictors. The book introduces List A (Section 4.1), List B (Section 5.2) and List C (Section 6.2), plus a few one-line lists inside single sections.

**N33 Firing set $F$ and $n$.** $F$ is the set of seeds whose rule fired: $F \subseteq \{0, 1, 2, 3, 4\}$. $n = \#F$ is an integer from 0 to 5. Seeds with `None` are dropped before anything is computed. If $n < 4$, the predictor is rejected automatically, whatever its other numbers are.

**N34 Lead and the 100-epoch margin.** Seed $i \in F$ leads if $P_i < G_i - 100$: the event comes more than 100 epochs before grok. 100 is the margin, an integer number of epochs.

**N35 The `seeds_before_grok` count.** A count in your saved results that calls a seed early if $G_i - P_i > 0$, not if $P_i < G_i - 100$. So it ignores the margin of **N34**. That's an audit finding.

**N36** lead_frac**, the lead fraction.** Scalar in $[0, 1]$: lead_frac $= \#\{i \in F : P_i < G_i - 100\}/n$, the share of firing seeds that lead.

**N37** $r_i$ **and** $\bar{r}$**, relative lead.** $r_i = (G_i - P_i)/G_i$ is a scalar for seed $i \in F$. Sign: $r_i > 0$ when the event is before grok, $r_i < 0$ when after; $r_i$ near 1 means the event is very early compared with grok. $\bar{r} = \frac{1}{n}\sum_{i \in F} r_i$ is the mean relative lead, a scalar.

**N38 Rank** $R(X_i)$ **and ties.** For a list of $n$ numbers $X_i$, $R(X_i)$ is the position of $X_i$ when the list is sorted from smallest to largest; the smallest has rank 1. A tie is two or more equal values; tied values all get the average of the positions they occupy.

**N39 Mean** $\bar{X}$ **and Pearson correlation** $C(X, Y)$**.** For two lists $X_i, Y_i$ of scalars over the same $n$ seeds, $\bar{X} = \frac{1}{n}\sum X_i$ (a scalar, the mean), and $C(X, Y) = \dfrac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum (X_i - \bar{X})^2 \sum (Y_i - \bar{Y})^2}}$, a scalar in $[-1, 1]$. It measures how close the points are to a straight line; $+1$ means a rising line. It is undefined if either list is constant (the denominator is 0); code then typically returns NaN, "not a number".

**N40 Spearman's** $\rho$**,** $d_i$**, fixed clock.** Scalar in $[-1, 1]$: $\rho = C\big(R(P), R(G)\big)$, the Pearson correlation of the ranks, over the $n$ seeds in $F$. $+1$ means later grok always comes with a later event. With no ties it equals $\rho = 1 - \dfrac{6\sum_{i \in F} d_i^2}{n(n^2 - 1)}$, where $d_i = R(G_i) - R(P_i)$ is an integer, the rank difference of seed $i$. A fixed clock is an event at about the same epoch in every run, whatever $G_i$ is; its $\rho$ is undefined (all events tied) or set by luck, by the order of tiny differences.

**N41 Adjacent swap.** Two seeds whose ranks are next to each other (e.g. 2 and 3) appear in the opposite order in $P$ and in $G$.

**N42** $n!$**, chance and p-value.** $n! = n \times (n - 1) \times \cdots \times 1$ is the number of orderings of $n$ items. "By chance" means all $n!$ orderings of the events are equally likely. The p-value of a $\rho$ is the probability, by chance, of a $\rho$ at least that large. A $\rho$ is called significant when its p-value is small.

## Part F. Judging predictors, and research honesty

**N43 PASS rule, FAIL, verdict.** A predictor gets the verdict PASS only if all three hold: lead_frac $\geq 0.8$, $\rho \geq 0.9$ and $0.05 \leq \bar{r} \leq 0.9$. Otherwise its verdict is FAIL. A predictor rejected automatically for $n < 4$ (**N33**) also gets FAIL. The function `evaluate_frozen_protocol` in `analyze_nanda_unified.py` (**A9**) applies this rule; scoring a predictor means running it. In the code the $\rho$ cutoff is written as 0.8999999999999998 so that a $\rho$ of 0.9 isn't lost to floating-point rounding.

**N44 Frozen protocol.** The fixed set of rules used to score every predictor: `nanda_unified.yaml`, $p = 113$, 5 seeds, 40,000 epochs, grok threshold 0.9, and the PASS rule (**N43**). It was written on 29 September 2026 and isn't changed while predictors are scored.

**N45 Closed-negative.** A predictor whose study is finished with a FAIL verdict. The first seven of your eight predictors are closed-negative.

**N46 Pre-registration, criterion validation, synthetic predictor, calibration, post-hoc.** Pre-registration: writing down the test before running it. A synthetic predictor is a made-up list of event epochs built from the known $G_i$, used to test the test. Criterion validation: checking that the PASS rule passes a good

synthetic predictor and fails a bad one. Calibration: setting a threshold of the test using data. Post-hoc: decided after the results were seen.

**N47 Look-ahead (hindsight) bias.** Using information from later epochs (e.g. $y_{\text{final}}$) to decide when an earlier event happened. A rule free of it could be run live during training.

**N48 Researcher degrees of freedom.** Choices of rule settings (**N30**), such as windows, thresholds or `skip_epochs`, made after looking at the results.

**N49 Sensitivity analysis and near-miss.** Sensitivity analysis: changing one setting a little and checking whether the verdict changes. A near-miss is a FAIL that misses a threshold by a small amount.

**N50 Baseline (control) and noise floor.** A control run is one where grokking doesn't happen, or a setting is switched off (e.g. $\lambda = 0$). The noise floor is how much a signal wobbles for no reason.

**N51 Negative result and reproducibility.** A negative result is a clear "this doesn't work". Reproducibility means someone else can rerun your work and get the same numbers.

**Module 1**

# The grokking phenomenon

## 1.1 Grokking

**Definition:** Grokking (**N24**) is what happens when, in one run, training accuracy $\text{acc}_{\text{train}}(t)$ reaches about 1 early, and much later test accuracy $\text{acc}_{\text{test}}(t)$ suddenly jumps to high values.

**Why it matters:** It's the event your whole benchmark tries to predict. Every predictor is judged by whether it signals before the jump, measured by the grok epoch $G_i$ (**N26**).

**Worked example:** In your runs, $\text{acc}_{\text{train}}(t)$ reaches 1 within a few hundred epochs, but $\text{acc}_{\text{test}}(t)$ only passes 0.9 at epochs 6921 to 23747, depending on the seed. There's no calculation here.

**Self-check:** (1) Why is grokking surprising? (The model already fits $D_{\text{train}}$, yet $\text{acc}_{\text{test}}$ keeps improving thousands of epochs later.) (2) Which two curves do you need to see it? ($\text{acc}_{\text{train}}(t)$ and $\text{acc}_{\text{test}}(t)$.)

**Likely viva question:** "Define grokking precisely. How is it different from ordinary slow learning?" (In slow learning $\text{acc}_{\text{train}}$ and $\text{acc}_{\text{test}}$ rise together. In grokking there's a long gap with $\text{acc}_{\text{train}} \approx 1$ and $\text{acc}_{\text{test}}$ low, then $\text{acc}_{\text{test}}$ jumps past 0.9 at $G_i$.)

## 1.2 Memorisation vs generalisation

**Definition:** Memorisation (**N23**) is getting the training pairs right by storing them: $\text{acc}_{\text{train}}$ is high while $\text{acc}_{\text{test}}$ stays low. Generalisation is getting new pairs right because the rule $(a + b) \bmod p$ has been learned: $\text{acc}_{\text{test}}$ is high too.

**Why it matters:** Grokking is the switch from memorisation to generalisation. So any warning must be about that switch. A signal that only reacts to memorisation tells you nothing about when the switch will come.

**Worked example:** The derivation uses only the five grok epochs to show that, at epoch 1,000, every run is still memorising.

**Derivation (H.K. Dass layout): every run is still memorising at epoch 1,000**

**Given:**

1. $(G_0, G_1, G_2, G_3, G_4) = (12987, 6921, 9510, 11019, 23747)$ (**N26**).
2. $\text{acc}_{\text{test}}(t) \leq 0.9$ at every epoch $t < G_i$, in the run of seed $i$ (**N26**).
3. $\text{acc}_{\text{train}}(t)$ reaches 1 within a few hundred epochs in every run (Section 1.1).

**Let:**

1. $G_i$: the grok epoch of seed $i$. Integer epoch, positive. Role: when that run starts to generalise.

2. $G_{\text{min}}$: the smallest of the five $G_i$. Integer epoch. Role: the earliest grok over all runs.
3. $t_1 = 1000$: the epoch being checked. Integer epoch.

**To find:** Whether $\text{acc}_{\text{test}}(t_1) \leq 0.9$ in every run.

**Solution:**

$$
\begin{aligned}
G_{\text{min}} &= \min\{12987, 6921, 9510, 11019, 23747\} && \because \text{definition of } G_{\text{min}} \text{ (Let 2) with Given 1} \\
&= 6921 && \because \text{seed 1 has the smallest value (\textbf{A2})} \\
t_1 = 1000 &< 6921 = G_{\text{min}} && \because \text{comparing the two numbers} \\
t_1 &< G_i \text{ for } i = 0, \ldots, 4 && \because \text{every } G_i \text{ is at least } G_{\text{min}} \\
\text{acc}_{\text{test}}(t_1) &\leq 0.9 \text{ in every run} && \because \text{Given 2}
\end{aligned}
$$

$\therefore$ At epoch 1,000, $\text{acc}_{\text{test}}$ is at most 0.9 in all five runs, while $\text{acc}_{\text{train}}$ has already reached 1 (Given 3). Every run is memorising, not yet generalising.

$$\boxed{\text{acc}_{\text{test}}(1000) \leq 0.9 \text{ in all five runs: memorising}}$$

**Self-check:** At epoch 8,000, which runs have generalised? (Only seed 1: $G_1 = 6921 < 8000$, and every other $G_i$ is above 8,000.)

**Likely viva question:** "Is your signal detecting the switch to generalisation, or just the end of memorisation?"

## 1.3 Circuit formation and cleanup

**Definition:** Nanda et al. [2] describe three phases (**N25**): memorisation; circuit formation, where a circuit that computes $(a+b)$ mod $p$ is built while $\text{acc}_{\text{test}}$ stays low; and cleanup, where the weights used for memorising are removed and $\text{acc}_{\text{test}}$ jumps.

**Why it matters:** It explains why a hidden signal could exist before $G_i$: the circuit is being built before the jump. That's the premise of the whole benchmark.

**Worked example:** Nanda report that cleanup happens at the same time as the jump in $\text{acc}_{\text{test}}$, so the circuit already exists before the jump. In your runs that jump is at $G_i$, between epochs 6921 and 23747. There's no calculation here.

**Self-check:** Which phase would a good early-warning signal need to detect? (Circuit formation, because it comes before cleanup and the jump.)

**Likely viva question:** "Nanda says the transition can't be predicted ahead of time. Why did you expect it could?"

**Module 2**

# Your training setup

## 2.1 Modular addition task

**Definition:** The model learns $c = (a + b) \bmod p$ **(N3)** for all inputs $a, b \in \{0, \ldots, p-1\}$, with $p = 113$ **(N1)**.

**Why it matters:** It's the standard grokking task of Power et al. [1] and Nanda et al. [2], so your results can be compared with theirs.

**Worked example:** The derivation counts the pairs and works out one answer.

**Derivation (H.K. Dass layout): number of pairs and one answer**

**Given:**

1. $p = 113$ (**N1**, `nanda_unified.yaml`).
2. Each of $a$ and $b$ takes $p$ values, $0$ to $p - 1$ **(N2)**.
3. Example inputs $a = 100$, $b = 50$.

**Let:**

1. $p$: the modulus. Integer, the same in every seed and epoch. Positive. Role: size of the task.
2. $N$: the number of all pairs. Integer, fixed before training. Positive. Role: the pool the split is cut from.
3. $a, b$: the inputs of one pair. Integers in $\{0, \ldots, 112\}$. Role: what the model reads.
4. $c$: the correct answer of that pair. Integer in $\{0, \ldots, 112\}$. Role: its label.

**To find:** $N$ and $c$ for $a = 100$, $b = 50$.

**Solution:**

$$
\begin{aligned}
N &= p \times p && \because\ p \text{ choices of } a \text{ times } p \text{ choices of } b \text{ (Given 2)} \\
&= 113 \times 113 && \because\ \text{substitution of Given 1} \\
&= 12769 && \because\ \text{multiplication} \\
c &= (a + b) \bmod p && \because\ \text{definition of } c \text{ (\textbf{N3})} \\
&= (100 + 50) \bmod 113 && \because\ \text{substitution of Given 1 and Given 3} \\
&= 150 \bmod 113 && \because\ \text{addition} \\
&= 150 - 1 \times 113 && \because\ 113 \text{ fits once into } 150 \text{ (\textbf{A4})} \\
&= 37 && \because\ \text{subtraction}
\end{aligned}
$$

$\therefore$ There are 12,769 pairs, and the pair $(100, 50)$ has the answer 37.

$$\boxed{N = 12769, \quad c = 37}$$

**Self-check:** How many possible answers does the model choose between? ($p = 113$: the integers 0 to 112, one logit each.)

**Likely viva question:** "Why $p = 113$ and not another prime?" (It matches Nanda et al. 2023 [2].)

## 2.2 Train/test split (30%)

**Definition:** A fraction $f = 0.3$ (**N6**) of the $N$ pairs forms the training set $D_{\text{train}}$; the other 70% form the test set $D_{\text{test}}$ (**N7**), which is never used to change the weights.

**Why it matters:** $G_i$ is measured on $D_{\text{test}}$, so the split must be fixed per seed and no test pair may leak into training. In your code the 30% is hard-coded in `get_dataloaders`; the config's `train_fraction` doesn't change it. That's an audit finding.

**Worked example:** The derivation computes the two set sizes.

**Derivation (H.K. Dass layout): sizes of the training and test sets**

**Given:**

1. $N = 12769$ (Section 2.1).
2. $f = 0.3$, hard-coded as `train_size = int(0.3 * len(...))` (**N6**).
3. `int()` of a positive number equals its floor (**A3**).

**Let:**

1. $N$: the number of all pairs. Integer, fixed before training. Role: the pool being split.
2. $f$: the train fraction. Scalar in $(0, 1)$. Role: share of pairs used for training.
3. $N_{\text{train}}$: the size of $D_{\text{train}}$. Integer, fixed per seed before training. Role: pairs used to change the weights.
4. $N_{\text{test}}$: the size of $D_{\text{test}}$. Integer, fixed per seed before training. Role: pairs that measure generalisation.

**To find:** $N_{\text{train}}$ and $N_{\text{test}}$.

**Solution:**

$$
\begin{aligned}
N_{\text{train}} &= \lfloor f \times N \rfloor && \because \texttt{int(0.3 * len(...))} \text{ and Given 3} \\
&= \lfloor 0.3 \times 12769 \rfloor && \because \text{substitution of Given 1 and Given 2} \\
&= \lfloor 3830.7 \rfloor && \because \text{multiplication} \\
&= 3830 && \because \text{floor drops the 0.7 (\textbf{A3})} \\
N_{\text{test}} &= N - N_{\text{train}} && \because \text{every pair not in } D_{\text{train}} \text{ is in } D_{\text{test}} \\
&= 12769 - 3830 && \because \text{substitution of Given 1 and the line above} \\
&= 8939 && \because \text{subtraction}
\end{aligned}
$$

$\therefore$ Each run trains on 3,830 pairs and tests on 8,939 pairs.

$$\boxed{N_{\text{train}} = 3830, \quad N_{\text{test}} = 8939}$$

**Self-check:** What goes wrong if a training pair appears in $D_{\text{test}}$? (The model can get it right by memorising, so $\text{acc}_{\text{test}}$ overstates generalisation and $G_i$ comes too early.)

**Likely viva question:** "How do you guarantee the test set is truly unseen, and that it's the same split when you reload a checkpoint?" (The seed fixes the split; rebuilding it with the same seed gives the same $D_{\text{test}}$.)

## 2.3 Transformer, logits and cross-entropy loss

**Definition:** The model (**N5**) is a one-layer, four-head transformer. For one pair it reads $(a, b)$ and the weights $\theta_t$ (**N10**) and returns the logits $z$ (**N11**), a vector with 113 entries, one score per possible answer. The softmax turns them into probabilities $q_k = e^{z_k} / \sum_{j=0}^{112} e^{z_j}$ (**N13**). The loss of the pair is the cross-entropy $\ell = -\ln q_c$ (**N14**), and the training loss $L(\theta_t)$ is the average of $\ell$ over $D_{\text{train}}$ (**N15**).

**Why it matters:** Every predictor is computed from this shared model's weights $\theta_t$ or its outputs.

**Worked example:** Two derivations: the loss at two probabilities, and why adding one constant to all logits leaves the loss unchanged.

**Derivation (H.K. Dass layout): the loss at $q_c = 0.25$ and at $q_c = 0.99$**

**Given:**

1. $\ell = -\ln q_c$ (**N14**).
2. Two example probabilities of the correct answer: $0.25$ and $0.99$.

**Let:**

1. $q_c$: the probability the model gives the correct answer $c$ of one pair, at $\theta_t$. Scalar in $(0, 1)$. Role: what the loss judges.
2. $\ell_{0.25}$: the loss of that pair when $q_c = 0.25$. Scalar $\geq 0$. Role: loss of a poor answer.
3. $\ell_{0.99}$: the loss of that pair when $q_c = 0.99$. Scalar $\geq 0$. Role: loss of a confident correct answer.

**To find:** $\ell_{0.25}$ and $\ell_{0.99}$.

**Solution:**

$$\begin{aligned}
\ell_{0.25} &= -\ln 0.25 && \because \text{ Given 1 with } q_c = 0.25 \\
&= -(-1.3863\ldots) && \because \ln 0.25 = -1.3863\ldots \\
&= 1.3863\ldots && \because -(-x) = x \\
\ell_{0.99} &= -\ln 0.99 && \because \text{ Given 1 with } q_c = 0.99 \\
&= -(-0.01005\ldots) && \because \ln 0.99 = -0.01005\ldots
\end{aligned}$$

$$= 0.01005\ldots \qquad \because\ -(-x) = x$$

$\therefore$ A probability of 0.25 on the correct answer costs about 1.39; a probability of 0.99 costs about 0.01. (Here "..." means the digits go on; the boxed values are rounded.)

$$\boxed{\ell_{0.25} = 1.39 \text{ (2 d.p.)}, \quad \ell_{0.99} = 0.01 \text{ (2 d.p.)}}$$

**Derivation (H.K. Dass layout): adding the same constant to all logits doesn't change the loss**

**Given:**

1. $q_k = e^{z_k} / \sum_{j=0}^{112} e^{z_j}$ (**N13**).
2. $\ell = -\ln q_c$ (**N14**).
3. $e^{x+y} = e^x e^y$ for scalars $x, y$ (rule of exponents).

**Let:**

1. $z$: the logits of one pair at $\theta_t$. Vector with 113 entries $z_0, \ldots, z_{112}$. Role: the model's scores.
2. $c$: the correct answer of that pair. Integer in $\{0, \ldots, 112\}$. Role: picks which probability the loss uses.
3. $s$: a constant. Scalar, any sign, the same for all 113 entries. Role: the shift being tested.
4. $z'$: the shifted logits, $z'_k = z_k + s$ for every $k$. Vector with 113 entries. Role: the changed input.
5. $q'_c$: the softmax probability of $c$ computed from $z'$. Scalar in $(0, 1)$.
6. $\ell'$: the loss computed from $z'$, $\ell' = -\ln q'_c$. Scalar $\geq 0$.

**To find:** Show that $\ell' = \ell$.

**Solution:**

$$\begin{aligned}
q'_c &= \frac{e^{z'_c}}{\sum_j e^{z'_j}} && \because \text{ Given 1 applied to } z' \\
&= \frac{e^{z_c+s}}{\sum_j e^{z_j+s}} && \because \text{ substitution } z'_k = z_k + s \text{ (Let 4)} \\
&= \frac{e^s e^{z_c}}{\sum_j e^{z_j+s}} && \because \text{ Given 3 on the numerator} \\
&= \frac{e^s e^{z_c}}{\sum_j e^s e^{z_j}} && \because \text{ Given 3 on each term of the denominator} \\
&= \frac{e^s e^{z_c}}{e^s \sum_j e^{z_j}} && \because\ e^s \text{ is the same scalar in every term, so it comes out of the sum} \\
&= \frac{e^{z_c}}{\sum_j e^{z_j}} && \because\ e^s/e^s = 1, \text{ since } e^s > 0 \\
&= q_c && \because \text{ Given 1} \\
\ell' &= -\ln q'_c && \because \text{ definition of } \ell' \text{ (Let 6)}
\end{aligned}$$

$$= -\ln q_c \qquad \because\ q'_c = q_c, \text{ shown above}$$
$$= \ell \qquad \because\ \text{Given 2}$$

$\therefore$ Adding the same number to all 113 logits leaves every probability, and so the loss, unchanged.

$$\boxed{\ell' = \ell}$$

**Self-check:** Why does adding the same constant to all 113 logits not change the loss? (The derivation above: $e^s$ cancels.)

**Likely viva question:** "Is your model exactly Nanda's?" (No. It lacks the attention output matrix $W_O$ (**N5**). That's a known gap.)

## 2.4 AdamW and weight decay

**Definition:** AdamW (**N18**) is the optimiser that turns gradients into weight updates. Weight decay $\lambda$ (**N19**) shrinks every weight a little at each update. The full update is the equation $\theta_{t+1} = \theta_t - \eta\, u_t - \eta\lambda\, \theta_t$ (**N20**).

**Why it matters:** You use $\lambda = 1.0$ with $\beta_1 = 0.9$, $\beta_2 = 0.98$, as in Nanda. Weight decay is believed to drive grokking. It also shrinks the weights on its own, in every run, so any signal computed from the size of the weights can move because of the optimiser rather than because grokking is coming. That's a confound (**N20**).

**Worked example:** The derivation rewrites the update to show the decay as a shrink factor.

**Derivation (H.K. Dass layout): the weight decay as a shrink factor**

**Given:**

1. $\theta_{t+1} = \theta_t - \eta\, u_t - \eta\lambda\, \theta_t$ (**N20**).
2. $\lambda = 1.0$ (**N19**, `nanda_unified.yaml`).

**Let:**

1. $\theta_t$: the weights after epoch $t$. Vector with $M$ entries. Role: what's being updated.
2. $\theta_{t+1}$: the weights after epoch $t+1$. Vector with $M$ entries.
3. $u_t$: the AdamW step at epoch $t$. Vector with $M$ entries, roughly uphill, so $-\eta u_t$ is downhill.
4. $\eta$: the learning rate. Scalar $> 0$.
5. $\lambda$: the weight decay. Scalar $\geq 0$.

**To find:** $\theta_{t+1}$ written as a scaled copy of $\theta_t$ plus the AdamW step, for $\lambda = 1.0$.

**Solution:**

$$\theta_{t+1} = \theta_t - \eta\, u_t - \eta\lambda\, \theta_t \qquad \because\ \text{Given 1}$$
$$= \theta_t - \eta\lambda\, \theta_t - \eta\, u_t \qquad \because\ \text{reordering the two subtracted terms}$$

$$
\begin{aligned}
&= (1 - \eta\lambda)\,\theta_t - \eta\,u_t && \because \text{ factoring the scalar out of } \theta_t - \eta\lambda\theta_t \\
&= (1 - \eta \times 1.0)\,\theta_t - \eta\,u_t && \because \text{ substitution of Given 2} \\
&= (1 - \eta)\,\theta_t - \eta\,u_t && \because \ \eta \times 1.0 = \eta
\end{aligned}
$$

$\therefore$ At every update the weights are first scaled by $(1 - \eta)$, which is below 1, and then moved by the AdamW step. The scaling pulls every weight towards 0 whatever the data does.

$$\boxed{\theta_{t+1} = (1 - \eta)\,\theta_t - \eta\,u_t}$$

**Self-check:** Why could a weight-based signal move because of the optimiser rather than because grokking is coming? (The factor $(1 - \eta)$ shrinks $\theta_t$ at every update, in every run.)

**Likely viva question:** "Did you vary $\lambda$? If not, how can you separate your signal from the effect of weight decay?"

## 2.5 Epochs and checkpoints

**Definition:** An epoch (**N9**) is one full pass over $D_{\text{train}}$. Your runs are full batch, so one epoch is one weight update. A checkpoint (**N21**) is a saved copy of $\theta_t$ at one epoch.

**Why it matters:** All runs last 40,000 epochs. $G_i$ and every event epoch are reported in epochs, and several predictors are computed afterwards from checkpoints. Checkpoints are saved every 50 epochs and evaluated every 100, so a quantity computed from checkpoint evaluations can only place an epoch to within 100 epochs.

**Worked example:** The derivation finds the two checkpoint evaluations around $G_0$, assuming evaluations fall on multiples of 100.

**Derivation (H.K. Dass layout): checkpoint evaluations around $G_0$**

**Given:**

1. $G_0 = 12987$ (**N26**).
2. Checkpoint evaluations every 100 epochs, taken here to be at multiples of 100 (**N21**).

**Let:**

1. $G_0$: the grok epoch of seed 0. Integer epoch.
2. $t_-$: the last evaluated epoch at or before $G_0$. Integer, a multiple of 100.
3. $t_+$: the first evaluated epoch after $G_0$. Integer, a multiple of 100.

**To find:** $t_-$ and $t_+$.

**Solution:**

$$t_- = 100 \times \left\lfloor \frac{G_0}{100} \right\rfloor \qquad \because \text{ largest multiple of 100 that's} \leq G_0 \text{ (\textbf{A3})}$$

$$
\begin{aligned}
&= 100 \times \lfloor 129.87 \rfloor && \because \text{ substitution of Given 1, and } 12987/100 = 129.87 \\
&= 100 \times 129 && \because \text{ floor} \\
&= 12900 && \because \text{ multiplication} \\
t_+ &= t_- + 100 && \because \text{ evaluations are 100 epochs apart (Given 2)} \\
&= 13000 && \because \text{ addition}
\end{aligned}
$$

$\therefore$ From checkpoint evaluations alone you'd only know that seed 0 groks between epochs 12,900 and 13,000.

$$\boxed{t_- = 12900, \quad t_+ = 13000}$$

Your $G_i$ values (e.g. 12987) aren't multiples of 50 or 100, so they come from an accuracy curve recorded more often than the checkpoints. Check which curve `analyze_nanda_unified.py` reads.

**Self-check:** Why must the data split be rebuilt with the same seed when re-reading a checkpoint? (Otherwise $D_{\text{test}}$ would contain pairs the checkpoint was trained on.)

**Likely viva question:** "What's your time resolution, and could it change any verdict?" (The lead margin of 100 epochs, Section 4.2, is the same size as the evaluation spacing.)

## 2.6 Seeds and why five

**Definition:** A seed $i$ (**N8**) fixes the random numbers of one run (the initial weights and the data split), so each seed is one independent, repeatable run.

**Why it matters:** Your benchmark uses 5 seeds, $i \in \{0, 1, 2, 3, 4\}$. Different seeds grok at different epochs, and that spread is what lets you test whether a predictor tracks when grok happens, not just that it happens.

**Worked example:** The derivation computes the mean grok epoch.

**Derivation (H.K. Dass layout): the mean grok epoch over the five seeds**

**Given:**

1. $(G_0, G_1, G_2, G_3, G_4) = (12987, 6921, 9510, 11019, 23747)$ (**N26**).
2. The mean of 5 numbers is their sum divided by 5.

**Let:**

1. $G_i$: the grok epoch of seed $i$. Integer epoch, positive.
2. $\bar{G}$: the mean grok epoch. Scalar epoch, positive. Role: a typical grok time.

**To find:** $\bar{G}$.

**Solution:**

$$\bar{G} = \frac{G_0 + G_1 + G_2 + G_3 + G_4}{5} \qquad \because \text{ Given 2}$$

$$= \frac{12987 + 6921 + 9510 + 11019 + 23747}{5} \qquad \because \text{ substitution of Given 1}$$

$$= \frac{64184}{5} \qquad \because \text{ addition}$$

$$= 12836.8 \qquad \because \text{ division}$$

$\therefore$ The five runs grok at epoch 12,836.8 on average, ranging from 6,921 (seed 1) to 23,747 (seed 4).

$$\boxed{\bar{G} = 12836.8}$$

**Self-check:** Why is one seed not enough to judge a predictor? (One run gives one grok epoch and one event epoch, so you can't tell whether the event moves with grok.)

**Likely viva question:** "Five is very few. What can and can't you conclude from five seeds?" (See Module 5.)

**Module 3**

# Measuring the outcome

## 3.1 Test accuracy

**Definition:** Test accuracy $\text{acc}_{\text{test}}(t)$ (**N22**) is the fraction of the 8,939 test pairs on which the predicted answer $\hat{k}$ (**N12**) equals the correct answer $c$, using $\theta_t$.

**Why it matters:** It's the only thing that defines grok in your benchmark.

**Worked example:** The derivation finds how many test pairs must be right for $\text{acc}_{\text{test}}$ to pass the grok threshold 0.9.

**Derivation (H.K. Dass layout): correct test pairs needed for** $\text{acc}_{\text{test}} > 0.9$

**Given:**

1. $N_{\text{test}} = 8939$ (Section 2.2).
2. $\text{acc}_{\text{test}}(t) = (\text{number of correct test pairs})/N_{\text{test}}$ (**N22**).
3. Grok needs $\text{acc}_{\text{test}}(t) > 0.9$ (**N26**).

**Let:**

1. $K$: the number of test pairs with $\hat{k} = c$ at $\theta_t$. Integer from 0 to 8939. Role: the count behind $\text{acc}_{\text{test}}$.
2. $K_{\text{min}}$: the smallest $K$ with $K/N_{\text{test}} > 0.9$. Integer.

**To find:** $K_{\text{min}}$.

**Solution:**

| | |
|---|---|
| $\frac{K}{N_{\text{test}}} > 0.9$ | $\because$ Given 2 put into Given 3 |
| $K > 0.9 \times N_{\text{test}}$ | $\because$ multiplying both sides by $N_{\text{test}} > 0$ |
| $K > 0.9 \times 8939$ | $\because$ substitution of Given 1 |
| $K > 8045.1$ | $\because$ multiplication |
| $K_{\text{min}} = 8046$ | $\because$ $K$ is an integer; 8046 is the first integer above 8045.1 |

$\therefore$ At least 8,046 of the 8,939 test pairs must be right. For example, 8,100 right gives $8100/8939 = 0.906$ (3 d.p.), which passes.

$$\boxed{K_{\text{min}} = 8046}$$

**Self-check:** Why not use test loss instead? (Loss also changes when the model just gets more or less confident on pairs it already gets right; accuracy only changes when answers change.)

**Likely viva question:** "Why accuracy and not loss to define grokking?"

## 3.2 The grok epoch $G_i$

**Definition:** The grok epoch of seed $i$ is $G_i = \min\{t : \text{acc}_{\text{test}}(t) > 0.9\}$ **(N26)**: the first epoch at which test accuracy goes above 0.9.

**Why it matters:** It's the target every predictor is compared with, and it's the same for all predictors. Recomputing it from the saved curves reproduces your saved values.

**Worked example:** $(G_0, G_1, G_2, G_3, G_4) = (12987, 6921, 9510, 11019, 23747)$. The derivation measures their spread.

**Derivation (H.K. Dass layout): the spread of the grok epochs**

**Given:**

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$ **(N26)**.

**Let:**

1. $G_{\text{max}}$: the largest $G_i$. Integer epoch.
2. $G_{\text{min}}$: the smallest $G_i$. Integer epoch.
3. $\Delta$: the range, $G_{\text{max}} - G_{\text{min}}$. Integer number of epochs, $\geq 0$. Role: how far apart the grok times are.

**To find:** $\Delta$.

**Solution:**

| | |
|---|---|
| $G_{\text{max}} = 23747$ | $\because$ seed 4 has the largest value in Given 1 |
| $G_{\text{min}} = 6921$ | $\because$ seed 1 has the smallest value in Given 1 |
| $\Delta = G_{\text{max}} - G_{\text{min}}$ | $\because$ definition of $\Delta$ (Let 3) |
| $= 23747 - 6921$ | $\because$ substitution of the two lines above |
| $= 16826$ | $\because$ subtraction |

$\therefore$ The grok epochs spread over 16,826 epochs. That spread is what a predictor's events must follow (Module 5).

$$\boxed{\Delta = 16826}$$

**Self-check:** Would $G_i$ change much with a 0.95 threshold? Why does it matter that the threshold is fixed? (It moves $G_i$ later by however long $\text{acc}_{\text{test}}$ takes to go from 0.9 to 0.95; fixing it stops it being picked to suit a predictor.)

**Likely viva question:** "Why 0.9? Is your result sensitive to that choice?"

**Module 4**

# What a predictor is

## 4.1 Predictor and event epoch $P_i$

**Definition:** A predictor (**N31**) is a (signal, rule) pair: a signal $y_t$ (**N27**) tracked during training, plus a rule (**N29**) that names one epoch $P_i$, the event epoch, as the warning in the run of seed $i$. If the rule never names an epoch, that's a miss, stored as `None`.

**Why it matters:** Each of your eight predictors is a (signal, rule) pair. If the rule or one of its settings (**N30**) changes, it's a different predictor, so every verdict belongs to one exact rule.

**Worked example:** Suppose some rule fires at the epochs below. This is **List A**, an illustrative event list (**N32**); it belongs to no predictor of yours. Modules 4 to 6 use it to show every calculation.

$$\text{List A:} \quad (P_0, P_1, P_2, P_3, P_4) = (3500,\ 6850,\ 2900,\ 4100,\ 7800).$$

All five seeds fire, so there's no `None`. There's no calculation here.

**Self-check:** What should the code do if the rule never fires? (Store `None`; the seed is then dropped from the firing set $F$, **N33**.)

**Likely viva question:** "Which exact rule is the official one for this predictor?" (Each predictor needs one named rule, with its settings, fixed before scoring. Two candidate rules are two predictors.)

## 4.2 Lead and the 100-epoch margin

**Definition:** A firing seed $i$ leads (**N34**) only if its event comes more than 100 epochs before grok: $P_i < G_i - 100$.

**Why it matters:** The margin stops an event that fires at grok, or just before it, from counting as a warning.

**Worked example:** The derivation checks seeds 0 and 1 of List A (Section 4.1). One leads; the other fires before grok but inside the margin.

**Derivation (H.K. Dass layout): do seeds 0 and 1 of List A lead?**

**Given:**

1. $G_0 = 12987$ and $G_1 = 6921$ (**N26**).
2. $P_0 = 3500$ and $P_1 = 6850$ (List A, Section 4.1).
3. Seed $i$ leads if $P_i < G_i - 100$ (**N34**).

**Let:**

1. $G_i$: the grok epoch of seed $i$. Integer epoch.

2. $P_i$: the event epoch of seed $i$ in List A. Scalar epoch.
3. $T_i$: the latest allowed event epoch for seed $i$, $T_i = G_i - 100$. Scalar epoch. Role: the cutoff an event must be below.

**To find:** Whether $P_0 < T_0$ and whether $P_1 < T_1$.

**Solution:**

$$
\begin{aligned}
T_0 &= G_0 - 100 && \because \text{ definition of } T_i \text{ (Let 3) with } i = 0 \\
&= 12987 - 100 && \because \text{ substitution of Given 1} \\
&= 12887 && \because \text{ subtraction} \\
P_0 = 3500 &< 12887 = T_0 && \because \text{ Given 2 and the line above: seed 0 leads} \\
T_1 &= G_1 - 100 && \because \text{ definition of } T_i \text{ (Let 3) with } i = 1 \\
&= 6921 - 100 && \because \text{ substitution of Given 1} \\
&= 6821 && \because \text{ subtraction} \\
P_1 = 6850 &> 6821 = T_1 && \because \text{ Given 2 and the line above: seed 1 doesn't lead}
\end{aligned}
$$

$\therefore$ Seed 0 leads. Seed 1 fires before grok ($6850 < 6921$), but only 71 epochs before, inside the 100-epoch margin, so it doesn't lead.

$$\boxed{P_0 < T_0 : \text{ seed 0 leads}; \quad P_1 > T_1 : \text{ seed 1 doesn't lead}}$$

**Self-check:** Does the `seeds_before_grok` count use the same margin? (No. It counts $G_i - P_i > 0$ (**N35**), so seed 1 of List A would count as early there but doesn't lead under the protocol. That's an audit finding.)

**Likely viva question:** "Why 100 epochs?"

## 4.3 Lead fraction

**Definition:** The lead fraction (**N36**) is the share of firing seeds that lead: lead_frac = $\#\{i \in F : P_i < G_i - 100\}/n$, where $F$ is the firing set and $n = \#F$ (**N33**).

**Why it matters:** PASS needs lead_frac $\geq 0.8$. With $n = 5$ that's at least 4 of 5 seeds; with $n = 4$ it's 4 of 4.

**Worked example:** The derivation computes lead_frac for List A, where all 5 seeds fire.

**Derivation (H.K. Dass layout):** lead_frac **of List A**

**Given:**

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$ (**N26**).
2. $(P_0, \ldots, P_4) = (3500, 6850, 2900, 4100, 7800)$ (List A, Section 4.1).
3. All 5 seeds fire, so $F = \{0, 1, 2, 3, 4\}$ (Given 2 has no `None`).
4. $(G_0 - 100, \ldots, G_4 - 100) = (12887, 6821, 9410, 10919, 23647)$ (subtracting 100 from each entry of Given 1).

5. lead_frac $= \#\{i \in F : P_i < G_i - 100\}/n$ **(N36)**.

**Let:**

1. $G_i$, $P_i$: grok epoch and event epoch of seed $i$. Scalars.
2. $n$: the number of firing seeds. Integer.
3. $\mathcal{L}$: the set of seeds in $F$ that lead. Set of seed labels. Role: the count on top of lead_frac.

**To find:** lead_frac.

**Solution:**

$$
\begin{aligned}
n &= \#F && \because \text{ definition of } n \text{ (N33)} \\
&= 5 && \because \text{ Given 3} \\
P_0 = 3500 &< 12887 = G_0 - 100 && \because \text{ Given 2 and 4: seed 0 leads} \\
P_1 = 6850 &> 6821 = G_1 - 100 && \because \text{ Given 2 and 4: seed 1 doesn't lead (Section 4.2)} \\
P_2 = 2900 &< 9410 = G_2 - 100 && \because \text{ Given 2 and 4: seed 2 leads} \\
P_3 = 4100 &< 10919 = G_3 - 100 && \because \text{ Given 2 and 4: seed 3 leads} \\
P_4 = 7800 &< 23647 = G_4 - 100 && \because \text{ Given 2 and 4: seed 4 leads} \\
\mathcal{L} &= \{0, 2, 3, 4\} && \because \text{ the four seeds that lead, from the lines above} \\
\#\mathcal{L} &= 4 && \because \text{ counting (A1)} \\
\text{lead_frac} &= \frac{\#\mathcal{L}}{n} && \because \text{ Given 5} \\
&= \frac{4}{5} && \because \text{ substitution of the lines above} \\
&= 0.8 && \because \text{ division}
\end{aligned}
$$

$\therefore$ Four of the five seeds lead, so List A sits exactly on the lead condition lead_frac $\geq 0.8$ and meets it.

$$\boxed{\text{lead_frac} = 0.8}$$

**Self-check:** If one seed misses and one fires after grok, what is lead_frac? (The miss is dropped, so $n = 4$, and 3 of those 4 lead: lead_frac $= 3/4 = 0.75$, a fail.)

**Likely viva question:** "Is 4 out of 5 strong evidence?"

## 4.4 Relative lead and mean relative lead

**Definition:** The relative lead of seed $i$ **(N37)** is $r_i = (G_i - P_i)/G_i$: how early the event is, as a fraction of the grok epoch. The mean relative lead is $\bar{r} = \frac{1}{n}\sum_{i \in F} r_i$.

**Why it matters:** PASS needs $0.05 \leq \bar{r} \leq 0.9$. Below 0.05 the event is too late to be useful. Above 0.9 it's so early that it's probably unrelated to grok.

**Worked example:** Two derivations: $r_0$ for List A, then $\bar{r}$ over all five seeds of List A.

**Derivation (H.K. Dass layout): relative lead of seed 0, List A**

**Given:**

1. $G_0 = 12987$ (**N26**).
2. $P_0 = 3500$ (List A, Section 4.1).
3. $r_i = (G_i - P_i)/G_i$ (**N37**).

**Let:**

1. $G_0$: grok epoch of seed 0. Integer epoch, positive.
2. $P_0$: event epoch of seed 0 in List A. Scalar epoch, positive.
3. $r_0$: relative lead of seed 0. Scalar; positive because $P_0 < G_0$.

**To find:** $r_0$.

**Solution:**

$$\begin{aligned} r_0 &= \frac{G_0 - P_0}{G_0} && \because \text{ Given 3 with } i = 0 \\ &= \frac{12987 - 3500}{12987} && \because \text{ substitution of Given 1 and Given 2} \\ &= \frac{9487}{12987} && \because \text{ subtraction} \\ &\approx 0.7305 && \because \text{ division, rounded to 4 d.p.} \end{aligned}$$

$\therefore$ Seed 0's event comes when only about 27% of the time to grok has passed, leaving about 73% of it as warning.

$$\boxed{r_0 = 0.7305 \text{ (4 d.p.)}}$$

**Derivation (H.K. Dass layout): mean relative lead of List A**

**Given:**

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$ (**N26**).
2. $(P_0, \ldots, P_4) = (3500, 6850, 2900, 4100, 7800)$ (List A, Section 4.1).
3. $n = 5$ and $F = \{0, 1, 2, 3, 4\}$ (Section 4.3).
4. $\bar{r} = \frac{1}{n} \sum_{i \in F} r_i$ with $r_i = (G_i - P_i)/G_i$ (**N37**).

**Let:**

1. $r_i$: relative lead of seed $i$. Scalar.
2. $\bar{r}$: mean relative lead. Scalar. Role: the third PASS condition.
3. $S$: the sum $\sum_{i=0}^{4} P_i/G_i$. Scalar, positive. Role: the part of $\bar{r}$ that depends on the events.

**To find:** $\bar{r}$.

**Solution:**

$$\begin{aligned} r_i &= \frac{G_i - P_i}{G_i} && \because \text{ Given 4} \\ &= \frac{G_i}{G_i} - \frac{P_i}{G_i} && \because \text{ splitting the fraction} \\ &= 1 - \frac{P_i}{G_i} && \because \ G_i/G_i = 1 \end{aligned}$$

$$\bar{r} = \frac{1}{5}\sum_{i=0}^{4}\left(1 - \frac{P_i}{G_i}\right) \qquad \because \text{Given 4 with Given 3, and the line above}$$

$$= \frac{1}{5}\left(5 - \sum_{i=0}^{4}\frac{P_i}{G_i}\right) \qquad \because \text{the five 1s add up to 5}$$

$$= 1 - \frac{1}{5}\sum_{i=0}^{4}\frac{P_i}{G_i} \qquad \because \text{multiplying out } \tfrac{1}{5}$$

$$= 1 - \frac{S}{5} \qquad \because \text{definition of } S \text{ (Let 3)}$$

$$S = \tfrac{3500}{12987} + \tfrac{6850}{6921} + \tfrac{2900}{9510} + \tfrac{4100}{11019} + \tfrac{7800}{23747} \qquad \because \text{Let 3 with Given 1 and 2}$$

$$\approx 0.26950 + 0.98974 + 0.30494 + 0.37208 + 0.32846 \qquad \because \text{each fraction rounded to 5 d.p.; later digits dropped}$$

$$= 2.26472 \qquad \because \text{addition}$$

$$\bar{r} = 1 - \frac{2.26472}{5} \qquad \because \text{substitution of } S \text{ into the line } \bar{r} = 1 - S/5$$

$$= 1 - 0.452944 \qquad \because \text{division by 5}$$

$$= 0.547056 \qquad \because \text{subtraction}$$

$\therefore$ The mean relative lead of List A is 0.547 to 3 d.p., inside $[0.05, 0.9]$, so List A meets this condition. Seed 1 contributes only $r_1 = 1 - 0.98974 = 0.01026$, because its event is just before grok.

$$\boxed{\bar{r} = 0.547 \text{ (3 d.p.)}, \quad 0.05 \le \bar{r} \le 0.9}$$

**Self-check:** What $\bar{r}$ does an event at exactly half the grok epoch give? ($P_i = 0.5\,G_i$ gives $r_i = 0.5$ in every seed, so $\bar{r} = 0.5$.)

**Likely viva question:** "Why have an upper limit at all?" (Without it, an event at epoch 1 in every run would score $\bar{r} \approx 1$ and look like the best possible warning.)

## 4.5 Misses and `None`

**Definition:** A miss (**N29**) is a seed where the rule never fires; the code stores `None`. Seeds with `None` are dropped before lead_frac, $\rho$ and $\bar{r}$ are computed (**N33**). If fewer than 4 seeds fire ($n < 4$), the predictor fails automatically.

**Why it matters:** How misses are handled changes lead_frac. Because a miss is dropped rather than counted as "no lead", four leads and one miss give lead_frac $= 4/4$, not $4/5$. That's a possible attack point for every predictor.

**Worked example:** The derivation works through one miss and four leads.

**Derivation (H.K. Dass layout): one miss and four leads**

**Given:**

1. One seed returns `None`; the other four fire and all four lead (illustrative case).
2. Seeds with `None` are dropped; $n < 4$ is an automatic fail (**N33**).
3. lead_frac = **#**$\{i \in F : P_i < G_i - 100\}/n$ (**N36**).

**Let:**

1. $F$: the firing set. Set of seed labels.
2. $n$: the number of firing seeds, $n = \#F$. Integer.
3. $\mathcal{L}$: the seeds in $F$ that lead. Set of seed labels.

**To find:** Whether the predictor fails automatically, and its lead_frac.

**Solution:**

$$
\begin{aligned}
n &= 5 - 1 && \because \text{ one of the 5 seeds is dropped (Given 1, 2)} \\
&= 4 && \because \text{ subtraction} \\
n &\geq 4 && \because \text{ so the automatic fail of Given 2 doesn't apply} \\
\#\mathcal{L} &= 4 && \because \text{ all four firing seeds lead (Given 1)} \\
\text{lead_frac} &= \frac{\#\mathcal{L}}{n} && \because \text{ Given 3} \\
&= \frac{4}{4} && \because \text{ substitution} \\
&= 1.0 && \because \text{ division}
\end{aligned}
$$

$\therefore$ The predictor isn't failed automatically, and its lead fraction is 1.0, the same as if all five seeds had led.

$$\boxed{n = 4, \quad \text{lead_frac} = 1.0}$$

**Self-check:** Can you compute $\rho$ with 4 seeds? (Yes: $\rho$ is then computed on the 4 firing seeds, with $n = 4$ in **N40**.)

**Likely viva question:** "How do you treat a seed where the signal never appears?"

**Module 5**

# Spearman's $\rho$ and small samples

## 5.1 Spearman rank correlation $\rho$

**Definition:** Spearman's $\rho$ (**N40**) is a number from $-1$ to $1$ that says whether two lists go up together in order, using only their ranks (**N38**). With no ties, $\rho = 1 - 6\sum_{i\in F} d_i^2/\big(n(n^2-1)\big)$, where $d_i = R(G_i) - R(P_i)$ is the rank difference of seed $i$.

**Why it matters:** PASS needs $\rho \geq 0.9$: seeds that grok later must also have later events. That shows the event tracks this run's grok, not a fixed clock (an event at about the same epoch in every run, whatever $G_i$ is).

**Worked example:** The derivation computes $\rho$ for List A (Section 4.1). Its five $P_i$ are all different, so there are no ties.

**Derivation (H.K. Dass layout): $\rho$ of List A**

**Given:**

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$ (**N26**).
2. $(P_0, \ldots, P_4) = (3500, 6850, 2900, 4100, 7800)$ (List A, Section 4.1).
3. $n = 5$ (Section 4.3).
4. Rank: position from smallest, starting at 1 (**N38**).
5. $\rho = 1 - 6\sum d_i^2/\big(n(n^2-1)\big)$ with $d_i = R(G_i) - R(P_i)$, when there are no ties (**N40**).

**Let:**

1. $R(G_i)$: rank of $G_i$ among the five grok epochs. Integer from 1 to 5.
2. $R(P_i)$: rank of $P_i$ among the five event epochs. Integer from 1 to 5.
3. $d_i$: rank difference of seed $i$. Integer; positive when seed $i$ ranks higher in $G$ than in $P$.
4. $\rho$: Spearman's rho of List A. Scalar in $[-1, 1]$.

**To find:** $\rho$.

**Solution:**

| | |
|---|---|
| sorted $G: 6921 < 9510 < 11019 < 12987 < 23747$ | $\because$ sorting Given 1: seeds 1, 2, 3, 0, 4 |
| $(R(G_0), \ldots, R(G_4)) = (4, 1, 2, 3, 5)$ | $\because$ position of each seed in the line above |
| sorted $P: 2900 < 3500 < 4100 < 6850 < 7800$ | $\because$ sorting Given 2: seeds 2, 0, 3, 1, 4 |
| $(R(P_0), \ldots, R(P_4)) = (2, 4, 1, 3, 5)$ | $\because$ position of each seed in the line above |
| $(d_0, \ldots, d_4) = (4-2,\ 1-4,\ 2-1,\ 3-3,\ 5-5)$ | $\because$ $d_i = R(G_i) - R(P_i)$ (Given 5) |
| $= (2, -3, 1, 0, 0)$ | $\because$ subtraction |
| $\sum d_i^2 = 2^2 + (-3)^2 + 1^2 + 0^2 + 0^2$ | $\because$ squaring each $d_i$ |

$$
\begin{aligned}
&= 4 + 9 + 1 + 0 + 0 && \because\ (-3)^2 = 9 \\
&= 14 && \because\ \text{addition} \\
\rho &= 1 - \frac{6 \sum d_i^2}{n(n^2-1)} && \because\ \text{Given 5} \\
&= 1 - \frac{6 \times 14}{5(5^2-1)} && \because\ \text{substitution of } \textstyle\sum d_i^2 = 14 \text{ and Given 3} \\
&= 1 - \frac{6 \times 14}{5 \times 24} && \because\ 5^2 - 1 = 25 - 1 = 24 \\
&= 1 - \frac{84}{5 \times 24} && \because\ 6 \times 14 = 84 \\
&= 1 - \frac{84}{120} && \because\ 5 \times 24 = 120 \\
&= 1 - 0.7 && \because\ 84/120 = 0.7 \\
&= 0.3 && \because\ \text{subtraction}
\end{aligned}
$$

$\therefore$ List A ranks the seeds with $\rho = 0.3$, far below 0.9, so it fails the $\rho$ condition. Its events don't follow the order of the grok epochs.

$$\boxed{\rho = 0.3 < 0.9}$$

**Self-check:** What $\rho$ does an event at a fixed epoch (the same in all seeds) get? (All $P_i$ tie, so every $R(P_i)$ is 3, the $R(P)$ list is constant, and $C$ in **N39** is undefined. Either way it can't reach 0.9, so it fails. Check whether your code returns NaN, "not a number", or 0 here.)

**Likely viva question:** "Why Spearman and not Pearson?"

## 5.2 Pearson vs Spearman

**Definition:** Pearson's correlation $C(X, Y)$ (**N39**) measures how close the points $(X_i, Y_i)$ are to a straight line, using the raw values. Spearman's $\rho$ is Pearson's correlation of the ranks, $\rho = C\big(R(P), R(G)\big)$, so it uses only order. When there are ties, this is how $\rho$ is computed.

**Why it matters:** With 5 seeds, seed 4 ($G_4 = 23747$) is far from the rest and dominates Pearson's $C$. Spearman isn't pulled by one extreme value, because 23747 only counts as rank 5. Any event list that rises with $G$ in exactly the same order has $\rho = 1$, even if the relationship is curved.

**Worked example:** Suppose a rule fires at the epochs of **List B**, an illustrative event list (**N32**) with a tie between seeds 0 and 3:

$$\text{List B:}\quad (P_0, P_1, P_2, P_3, P_4) = (4000,\ 2000,\ 3000,\ 4000,\ 9000).$$

The derivation computes its $\rho$ with the tie.

**Derivation (H.K. Dass layout): $\rho$ of List B, with a tie**

**Given:**

1. $(P_0, \ldots, P_4) = (4000, 2000, 3000, 4000, 9000)$ (List B, above).

2. $(R(G_0), \ldots, R(G_4)) = (4, 1, 2, 3, 5)$ (Section 5.1).
3. Tied values get the average of the positions they occupy (**N38**).
4. $\rho = C\big(R(P), R(G)\big)$, with $C$ from **N39**; $n = 5$.

**Let:**

1. $X_i = R(P_i)$: rank of the event of seed $i$. Scalar (can be a half).
2. $Y_i = R(G_i)$: rank of the grok epoch of seed $i$. Integer.
3. $\bar{X}$, $\bar{Y}$: the means of the $X_i$ and of the $Y_i$. Scalars.
4. $\rho$: Spearman's rho with the tie. Scalar in $[-1, 1]$.

**To find:** $\rho$.

**Solution:**

$$\text{sorted } P : 2000 < 3000 < 4000 = 4000 < 9000 \quad \because \text{ sorting Given 1: seeds 1, 2, then 0 and 3 tied, then 4}$$

$$(X_0, \ldots, X_4) = (3.5, 1, 2, 3.5, 5) \quad \because \text{ seeds 0 and 3 tie for positions 3 and 4 (Given 3)}$$

$$\bar{X} = \frac{3.5 + 1 + 2 + 3.5 + 5}{5} \quad \because \text{ definition of the mean (**N39**)}$$

$$= \frac{15}{5} \quad \because \text{ addition}$$

$$= 3 \quad \because \text{ division}$$

$$\bar{Y} = \frac{4 + 1 + 2 + 3 + 5}{5} \quad \because \text{ Given 2 and the definition of the mean}$$

$$= 3 \quad \because \ 15/5 = 3$$

$$(X_i - \bar{X}) = (0.5, -2, -1, 0.5, 2) \quad \because \text{ subtracting } \bar{X} = 3$$

$$(Y_i - \bar{Y}) = (1, -2, -1, 0, 2) \quad \because \text{ subtracting } \bar{Y} = 3$$

$$\sum (X_i - \bar{X})(Y_i - \bar{Y}) = 0.5 + 4 + 1 + 0 + 4 \quad \because \text{ multiplying the two lines above entry by entry}$$

$$= 9.5 \quad \because \text{ addition}$$

$$\sum (X_i - \bar{X})^2 = 0.25 + 4 + 1 + 0.25 + 4 \quad \because \text{ squaring each entry}$$

$$= 9.5 \quad \because \text{ addition}$$

$$\sum (Y_i - \bar{Y})^2 = 1 + 4 + 1 + 0 + 4 \quad \because \text{ squaring each entry}$$

$$= 10 \quad \because \text{ addition}$$

$$\rho = \frac{9.5}{\sqrt{9.5 \times 10}} \quad \because \text{ Given 4 with the three sums above}$$

$$= \frac{9.5}{\sqrt{95}} \quad \because \text{ multiplication}$$

$$\approx 0.975 \quad \because \ \sqrt{95} = 9.7468\ldots, \text{ division, rounded to 3 d.p.}$$

$\therefore$ With the tie, $\rho = 0.975$. If seed 0's event were a little later than seed 3's, the ranks would be $(4, 1, 2, 3, 5)$ and $\rho = 1$; if a little earlier, seeds 0 and 3 would swap and $\rho = 0.9$. So a tie gives a value between the two, outside the steps of Section 5.3. Ties change the answer, which is why they matter.

$$\boxed{\rho = 0.975 \text{ (3 d.p.)}}$$

**Self-check:** Which one would seed 4 distort, and why? (Pearson's $C$ on raw values: $G_4 = 23747$ is far above the others, so it dominates the sums in **N39**.)

**Likely viva question:** "Your Pearson values look strong; why don't you trust them?" (With five seeds, one extreme seed can decide Pearson's value, even its sign. On ranks, that seed can move the result by at most its rank.)

## 5.3 Only 21 possible values with 5 seeds

**Definition:** With $n = 5$ and no ties, $\rho = 1 - \sum d_i^2/20$, and $\sum d_i^2$ is an even number from 0 to 40. So $\rho$ can only be $-1, -0.9, \ldots, 0.9, 1$: 21 values, in steps of 0.1.

**Why it matters:** $\rho \geq 0.9$ means "perfect order, or exactly one adjacent swap" (**N41**). It also explains the coded cutoff 0.8999999999999998 (**N43**): it keeps a $\rho$ of exactly 0.9 from failing through floating-point rounding. The criterion-validation record in `context.md` says $\rho$ has only 11 values in steps of 0.2; that's wrong, and the list it gives leaves out 0.9.

**Worked example:** The derivation finds the formula for $n = 5$, why $\sum d_i^2$ is even, its largest value, and the $\rho$ of one adjacent swap.

**Derivation (H.K. Dass layout): the possible values of $\rho$ with five seeds**

**Given:**

1. $\rho = 1 - 6\sum d_i^2/\big(n(n^2-1)\big)$, no ties (**N40**).
2. $n = 5$, and each list of ranks is $1, 2, 3, 4, 5$ in some order (**N38**).
3. $d_i = R(G_i) - R(P_i)$ is an integer (**N40**).

**Let:**

1. $D$: the sum of squared rank differences, $D = \sum_i d_i^2$. Integer $\geq 0$. Role: the only thing $\rho$ depends on.
2. $D_{\max}$: the largest possible $D$. Integer.
3. $\rho_{\text{swap}}$: the $\rho$ when exactly one adjacent pair is swapped. Scalar.

**To find:** The formula for $\rho$ at $n = 5$, the possible $D$, and $\rho_{\text{swap}}$.

**Solution:**

$$\rho = 1 - \frac{6D}{5(5^2-1)} \qquad \because \text{Given 1 with } n = 5 \text{ (Given 2) and Let 1}$$

$$= 1 - \frac{6D}{5 \times 24} \qquad \because 5^2 - 1 = 24$$

$$= 1 - \frac{6D}{120} \qquad \because 5 \times 24 = 120$$

$$= 1 - \frac{D}{20} \qquad \because \text{dividing top and bottom by 6}$$

$$\sum_i d_i = \sum_i R(G_i) - \sum_i R(P_i) \qquad \because \text{Given 3, summed over the 5 seeds}$$

$$
\begin{aligned}
&= 15 - 15 && \because \text{ each rank list adds up to } 1+2+3+4+5 = 15 \text{ (Given 2)} \\
&= 0 && \because \text{ subtraction} \\
d_i^2 - d_i &= d_i(d_i - 1) && \because \text{ factoring out } d_i \\
D - \sum_i d_i &= \sum_i d_i(d_i - 1) && \because \text{ summing the line above over the 5 seeds} \\
D - 0 &= \sum_i d_i(d_i - 1) && \because \textstyle\sum_i d_i = 0, \text{ shown above} \\
D &= \text{an even number} && \because \ d_i(d_i - 1) \text{ is a product of two consecutive integers, so even} \\
D_{\max} &= 4^2 + 2^2 + 0^2 + (-2)^2 + (-4)^2 && \because \text{ fully reversed order, } d = (4, 2, 0, -2, -4) \\
&= 16 + 4 + 0 + 4 + 16 && \because \text{ squaring} \\
&= 40 && \because \text{ addition} \\
\#\{0, 2, 4, \ldots, 40\} &= \frac{40}{2} + 1 && \because \text{ even numbers from 0 to 40} \\
&= 21 && \because \text{ arithmetic} \\
\rho_{\text{swap}} &= 1 - \frac{1^2 + (-1)^2}{20} && \because \text{ one adjacent swap: two seeds have } d_i = \pm 1, \text{ the rest 0} \\
&= 1 - \frac{2}{20} && \because \ 1^2 = (-1)^2 = 1, \text{ addition} \\
&= 0.9 && \because \ 2/20 = 0.1, \text{ subtraction}
\end{aligned}
$$

$\therefore$ With five seeds $\rho = 1 - D/20$ with $D$ even from 0 to 40; listing all 120 orderings shows every even value occurs, so $\rho$ has 21 possible values. One adjacent swap gives exactly 0.9; two separate adjacent swaps give $D = 4$ and $\rho = 0.8$, a fail. (List A in Section 5.1 has $D = 14$, so $\rho = 1 - 14/20 = 0.3$, the same answer.)

$$\boxed{\rho = 1 - \frac{D}{20}, \quad 21 \text{ values}, \quad \rho_{\text{swap}} = 0.9}$$

**Self-check:** Can $\rho = 0.85$ happen with 5 seeds and no ties? (No: it would need $D = 3$, which is odd.)

**Likely viva question:** "Isn't the $\rho$ test very coarse with five seeds?"

## 5.4 Significance with small $n$

**Definition:** Significance (**N42**) asks whether a correlation could easily appear by chance, that is, if all $n!$ orderings of the events were equally likely. The p-value is the probability, by chance, of a $\rho$ at least as large as the one observed.

**Why it matters:** With $n = 5$, only $\rho = 1$, and at the edge $\rho = 0.9$, are unlikely by chance. Smaller values, such as List A's $\rho = 0.3$, are easy to get by chance.

**Worked example:** The derivation computes the chance of $\rho = 1$ and of $\rho \geq 0.9$.

**Derivation (H.K. Dass layout): chance probabilities of $\rho = 1$ and $\rho \geq 0.9$**

**Given:**

1. $n = 5$; by chance, all $n!$ orderings are equally likely (**N42**).
2. $\rho = 1$ only for the one ordering with $D = 0$; $\rho = 0.9$ only for orderings with exactly one adjacent swap (Section 5.3).
3. With 5 positions there are 4 adjacent pairs: (1,2), (2,3), (3,4), (4,5).

**Let:**

1. $n!$: the number of orderings of 5 seeds. Integer.
2. $\Pr(\rho = 1)$: chance probability of $\rho = 1$. Scalar in $[0, 1]$.
3. $\Pr(\rho \geq 0.9)$: chance probability of $\rho \geq 0.9$, the p-value of $\rho = 0.9$. Scalar in $[0, 1]$.

**To find:** $\Pr(\rho = 1)$ and $\Pr(\rho \geq 0.9)$.

**Solution:**

$$
\begin{aligned}
n! &= 5 \times 4 \times 3 \times 2 \times 1 && \because \text{ definition of } n! \text{ with } n = 5 \\
&= 120 && \because \text{ multiplication} \\
\Pr(\rho = 1) &= \frac{1}{120} && \because \text{ one ordering out of 120 (Given 1, 2)} \\
\Pr(\rho \geq 0.9) &= \frac{1 + 4}{120} && \because \text{ the perfect order plus 4 single adjacent swaps (Given 2, 3)} \\
&= \frac{5}{120} && \because \text{ addition} \\
&= \frac{1}{24} && \because \text{ dividing top and bottom by 5} \\
&\approx 0.042 && \because \text{ division, rounded to 3 d.p.}
\end{aligned}
$$

$\therefore$ A random event list would reach $\rho = 1$ once in 120 tries, and $\rho \geq 0.9$ about once in 24.

$$\boxed{\Pr(\rho = 1) = \tfrac{1}{120}, \quad \Pr(\rho \geq 0.9) = \tfrac{1}{24}}$$

**Self-check:** Why does your protocol use a threshold rather than a p-value? (With 120 orderings the p-values are coarse too; a fixed threshold is simpler to pre-register. By the same count, 14 of the 120 orderings give $\rho \geq 0.7$, a p-value of about 0.12.)

**Likely viva question:** "How confident can you be in a PASS from five points?"

**Module 6**

# The frozen protocol and the PASS rule

## 6.1 The frozen protocol

**Definition:** The frozen protocol (**N44**) is the fixed set of rules used to score every predictor: the config `nanda_unified.yaml`, $p = 113$, 5 seeds, 40,000 epochs, grok threshold 0.9, and the PASS rule of Section 6.2. It's written down and not changed while predictors are scored.

**Why it matters:** It makes all predictors comparable, and it stops thresholds being tuned to make a favourite predictor pass. It was written on 29 September 2026. Any result that was already known by that date weakens the claim that the thresholds were chosen blind, so the dates in `context.md` matter.

**Worked example:** The verdict function `evaluate_frozen_protocol` in `analyze_nanda_unified.py` (**A9**) applies the three conditions of Section 6.2. There's no calculation here.

**Self-check:** Does every predictor have a saved frozen-protocol verdict in a results file, or were some only recomputed later? (Check the saved results; a verdict that was never saved can't be checked by an examiner.)

**Likely viva question:** "Was the protocol fixed before you saw any results?"

## 6.2 The PASS rule

**Definition:** A predictor passes (**N43**) only if all three hold:

$$\text{lead_frac} \geq 0.8, \qquad \rho \geq 0.9, \qquad 0.05 \leq \bar{r} \leq 0.9.$$

Otherwise its verdict is FAIL.

**Why it matters:** Each condition blocks a different failure: lead_frac blocks firing late, $\rho$ blocks firing on a fixed clock, and the upper limit on $\bar{r}$ blocks firing absurdly early.

**Worked example:** List A (Section 4.1): lead_frac $= 0.8$ (passes, Section 4.3), $\rho = 0.3$ (fails, Section 5.1), $\bar{r} = 0.547$ (passes, Section 4.4), so FAIL. One failed condition is enough. The derivation scores a second illustrative list. Suppose a rule fires at the epochs of **List C** (**N32**), all close to epoch 2000:

$$\text{List C:} \quad (P_0, P_1, P_2, P_3, P_4) = (2020,\ 1990,\ 2000,\ 2030,\ 2010).$$

**Derivation (H.K. Dass layout): the verdict of List C**

**Given:**

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$ and $(R(G_0), \ldots, R(G_4)) = (4, 1, 2, 3, 5)$ (**N26**, Section 5.1).

2. $(P_0, \ldots, P_4) = (2020, 1990, 2000, 2030, 2010)$, all five fire (List C, above).
3. Every $P_i$ is at most 2030 and every $G_i - 100$ is at least $6921 - 100 = 6821$, so all five seeds lead (**N34**).
4. $\rho = 1 - D/20$ for $n = 5$ with no ties (Section 5.3).
5. $\bar{r} = 1 - \frac{1}{5}\sum_{i=0}^{4} P_i/G_i$ when $n = 5$ (Section 4.4).
6. PASS rule (**N43**).

**Let:**

1. $R(P_i)$: rank of $P_i$ among the five events. Integer from 1 to 5. $R(P)$ is the list $(R(P_0), \ldots, R(P_4))$.
2. $d_i = R(G_i) - R(P_i)$: rank difference of seed $i$. Integer. $d$ is the list $(d_0, \ldots, d_4)$.
3. $D = \sum d_i^2$. Integer.
4. lead_frac, $\rho$, $\bar{r}$: the three PASS quantities of List C. Scalars.
5. $S$: the sum $\sum_{i=0}^{4} P_i/G_i$ for List C. Scalar, positive.

**To find:** The three PASS quantities and the verdict.

**Solution:**

$$\begin{aligned}
\text{lead_frac} &= \frac{5}{5} && \because \text{ Given 3, } n = 5 \\
&= 1.0 && \because \text{ division} \\
\text{sorted } P &: 1990 < 2000 < 2010 < 2020 < 2030 && \because \text{ sorting Given 2: seeds 1, 2, 4, 0, 3} \\
R(P) &= (4, 1, 2, 5, 3) && \because \text{ position of each seed in the line above} \\
d &= (4-4,\ 1-1,\ 2-2,\ 3-5,\ 5-3) && \because \text{ Let 2 with Given 1} \\
&= (0, 0, 0, -2, 2) && \because \text{ subtraction} \\
D &= 0^2 + 0^2 + 0^2 + (-2)^2 + 2^2 && \because \text{ Let 3} \\
&= 0 + 0 + 0 + 4 + 4 && \because \text{ squaring} \\
&= 8 && \because \text{ addition} \\
\rho &= 1 - \frac{8}{20} && \because \text{ Given 4} \\
&= 0.6 && \because 8/20 = 0.4\text{, subtraction} \\
\bar{r} &= 1 - \frac{S}{5} && \because \text{ Given 5 and Let 5} \\
S &= \tfrac{2020}{12987} + \tfrac{1990}{6921} + \tfrac{2000}{9510} + \tfrac{2030}{11019} + \tfrac{2010}{23747} && \because \text{ Let 5 with Given 1 and 2} \\
&\approx 0.15554 + 0.28753 + 0.21030 + 0.18423 + 0.08464 && \because \text{ each fraction rounded to 5 d.p.; later digits dropped} \\
&= 0.92224 && \because \text{ addition} \\
\bar{r} &= 1 - \frac{0.92224}{5} && \because \text{ substitution of } S \text{ into the line } \bar{r} = 1 - S/5 \\
&= 1 - 0.184448 && \because \text{ division by 5} \\
&= 0.815552 && \because \text{ subtraction}
\end{aligned}$$

$\therefore$ List C has lead_frac $= 1.0$ (passes), $\rho = 0.6$ (fails) and $\bar{r} = 0.816$ to 3 d.p. (passes). One failed condition is enough, so the verdict is FAIL. List C fires at about epoch 2000 in

every seed whatever $G_i$ is: it's close to a fixed clock, and its $\rho$ of 0.6 comes only from the order of differences of a few tens of epochs.

$$\boxed{\text{lead_frac} = 1.0,\ \rho = 0.6,\ \bar{r} = 0.816 :\ \text{FAIL}}$$

**Self-check:** Build a made-up event list that passes all three for your five $G_i$. (Section 6.3 does this with $P_i = 0.5\,G_i$.)

**Likely viva question:** "Justify each threshold. Where do 0.8, 0.9, 0.05 and 0.9 come from?"

## 6.3 Pre-registration and criterion validation

**Definition:** Pre-registration (**N46**) is writing down your test before running it. Criterion validation is checking that the test passes a good synthetic predictor and fails a bad one, where a synthetic predictor is a made-up list of event epochs built from your $G_i$.

**Why it matters:** Your `context.md` records a calibration of the $\rho$ cutoff. It used this run's own grok epochs, after some results were already known, so an examiner can call it post-hoc.

**Worked example:** The derivation scores the synthetic "perfect" predictor $P_i = 0.5\,G_i$.

**Derivation (H.K. Dass layout): the synthetic predictor** $P_i = 0.5\,G_i$

**Given:**

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$, all different (**N26**).
2. Synthetic events $P_i = 0.5\,G_i$ for every seed, so all five fire.
3. Lead: $P_i < G_i - 100$ (**N34**); $r_i = (G_i - P_i)/G_i$ (**N37**).
4. PASS rule (**N43**).

**Let:**

1. $P_i$: synthetic event epoch of seed $i$. Scalar epoch.
2. $r_i$: its relative lead. Scalar.
3. lead_frac, $\rho$, $\bar{r}$: the three PASS quantities. Scalars.

**To find:** The three PASS quantities and the verdict.

**Solution:**

$$\begin{aligned}
G_i - P_i &= G_i - 0.5\,G_i && \because \text{Given 2} \\
&= 0.5\,G_i && \because \text{subtraction} \\
&\geq 0.5 \times 6921 && \because \text{the smallest } G_i \text{ is 6921 (Given 1)} \\
&= 3460.5 && \because \text{multiplication} \\
&> 100 && \because \text{comparison, so } P_i < G_i - 100 \text{ in every seed}
\end{aligned}$$

$$
\begin{aligned}
\text{lead_frac} &= \frac{5}{5} && \because \text{ all five seeds lead (Given 3, line above)} \\
&= 1.0 && \because \text{ division} \\
R(P_i) &= R(G_i) && \because \ P_i = 0.5\,G_i \text{ keeps the order of the } G_i \\
d_i &= 0 && \because \ d_i = R(G_i) - R(P_i) \\
\rho &= 1 - \frac{0}{20} && \because \text{ Section 5.3 with } D = 0 \\
&= 1 && \because \text{ arithmetic} \\
r_i &= \frac{0.5\,G_i}{G_i} && \because \text{ Given 3 and the second line} \\
&= 0.5 && \because \ G_i/G_i = 1 \\
\bar{r} &= \frac{5 \times 0.5}{5} && \because \text{ all five } r_i \text{ equal 0.5} \\
&= 0.5 && \because \text{ division}
\end{aligned}
$$

$\therefore$ The perfect synthetic predictor gets lead_frac $= 1.0$, $\rho = 1$ and $\bar{r} = 0.5$, so it must PASS. A synthetic event at a fixed epoch, e.g. $P_i = 2000$ in every seed, has all events tied, so $\rho$ is undefined (Section 5.1) and can't reach 0.9; it must FAIL.

$$\boxed{\text{lead_frac} = 1.0,\ \rho = 1,\ \bar{r} = 0.5 :\ \text{PASS}}$$

**Self-check:** What would a synthetic control that should fail look like? (The fixed $P_i = 2000$ above; or an event placed after grok, $P_i = G_i + 100$, which has no leads.)

**Likely viva question:** "How do you know your criterion isn't too strict to pass anything?" (The perfect synthetic predictor passes it.)

**Module 7**

# Research honesty

## 7.1 Hindsight (look-ahead) bias

**Definition:** Look-ahead bias (**N47**) is using information from later epochs, such as the final value $y_{\text{final}}$ of a signal at epoch 40,000 (**N27**), to decide when an earlier event happened.

**Why it matters:** A rule with look-ahead can pass the PASS rule and still be useless, because a live monitor couldn't run it during training: the information it needs doesn't exist yet. Any rule that uses $y_{\text{final}}$, the run's largest or smallest value, or an average over the whole run, has this problem, so such a result can only be reported as a hindsight check, not as a benchmark result.

**Worked example:** Suppose a rule fires at the first epoch where the signal falls below its average over the whole run. The derivation shows that this average needs epochs after the event.

**Derivation (H.K. Dass layout): a whole-run average needs future epochs**

**Given:**

1. A signal $y_t$, recorded here at every epoch $t = 0, 1, \ldots, 40{,}000$, so $40{,}001$ values (**N27**, **N9**; illustrative).
2. The rule compares $y_t$ with the whole-run average $\bar{y}_{\text{run}}$ and fires at the first epoch where $y_t < \bar{y}_{\text{run}}$ (illustrative rule).
3. The candidate event epoch $\tau$ is before the end of the run: $\tau < 40{,}000$.

**Let:**

1. $y_t$: the signal at epoch $t$. Scalar.
2. $\bar{y}_{\text{run}}$: the whole-run average, $\bar{y}_{\text{run}} = \frac{1}{40001}\sum_{t=0}^{40000} y_t$. Scalar. Role: the level the rule compares against.
3. $\tau$: an epoch at which the rule might fire. Integer epoch.

**To find:** $\bar{y}_{\text{run}}$ split into a past part (epochs up to $\tau$) and a future part (epochs after $\tau$).

**Solution:**

$$\begin{aligned}
\bar{y}_{\text{run}} &= \frac{1}{40001}\sum_{t=0}^{40000} y_t && \because \text{ definition of } \bar{y}_{\text{run}} \text{ (Let 2)} \\
&= \frac{1}{40001}\left(\sum_{t=0}^{\tau} y_t + \sum_{t=\tau+1}^{40000} y_t\right) && \because \text{ splitting the sum at } \tau \\
&= \frac{1}{40001}\sum_{t=0}^{\tau} y_t + \frac{1}{40001}\sum_{t=\tau+1}^{40000} y_t && \because \text{ multiplying out } \tfrac{1}{40001}
\end{aligned}$$

$\therefore$ The second term adds up the $40{,}000 - \tau$ values after $\tau$, and $40{,}000 - \tau \geq 1$ by Given 3. So $\bar{y}_{\text{run}}$ isn't known at epoch $\tau$: the rule looks ahead. For $\tau = 3000$, 37,000 of the 40,001 values are from the future.

$$\boxed{\bar{y}_{\text{run}} \text{ uses } y_{\tau+1}, \ldots, y_{40000}, \text{ which come after } \tau}$$

**Self-check:** How would you test whether a rule is look-ahead free? (Cut the curve at epoch $t$, rerun the rule, and check that the event before $t$ doesn't move.)

**Likely viva question:** "Could this signal be used during training, before you know the outcome?"

## 7.2 Causal vs centred moving average

**Definition:** A causal moving average at epoch $t$ (**N28**) uses only points at or before $t$. A centred one uses points on both sides of $t$.

**Why it matters:** A centred average includes later epochs in each smoothed value, so any event found on a centred curve is slightly look-ahead. `uniform_filter1d` is centred (with the edges padded by copying). A rule that smooths with it can't be called an ex-ante trigger (one that uses only the past).

**Worked example:** The derivation writes both averages for a window of 3 points.

**Derivation (H.K. Dass layout): causal and centred averages, window 3**

**Given:**

1. A signal $y_t$, one scalar per point $t$ (**N27**).
2. Window $w = 3$; causal uses points $\leq t$, centred uses both sides (**N28**).

**Let:**

1. $y_t$: the signal at point $t$. Scalar.
2. $\bar{y}_t^{\text{causal}}$: the causal average at $t$. Scalar. Uses $y_{t-2}, y_{t-1}, y_t$.
3. $\bar{y}_t^{\text{centred}}$: the centred average at $t$. Scalar. Uses $y_{t-1}, y_t, y_{t+1}$.

**To find:** Both averages, and which needs a later point.

**Solution:**

$$\bar{y}_t^{\text{causal}} = \frac{y_{t-2} + y_{t-1} + y_t}{3} \qquad \because \text{ the 3 points ending at } t \text{ (Given 2)}$$

$$\bar{y}_t^{\text{centred}} = \frac{y_{t-1} + y_t + y_{t+1}}{3} \qquad \because \text{ one point each side of } t \text{ (Given 2)}$$

$\therefore$ The centred average at $t$ contains $y_{t+1}$, a point from the future; the causal one doesn't.

$$\boxed{\bar{y}_t^{\text{centred}} \text{ needs } y_{t+1};\ \bar{y}_t^{\text{causal}} \text{ doesn't}}$$

**Self-check:** Which one shifts events later? (The causal one: it only reacts after the change has happened, so its events come later than the centred version's.)

**Likely viva question:** "Is your smoothing causal?" (Check which function each rule uses; `uniform_filter1d` is centred.)

## 7.3 Researcher degrees of freedom

**Definition:** Researcher degrees of freedom (**N48**) are choices of rule settings (**N30**), such as windows, thresholds or `skip_epochs`, made after looking at the results.

**Why it matters:** Each extra version of a rule that you try is another chance to pass by luck. If settings were changed after looking at the curves, an examiner can say the rule was fitted to the data.

**Worked example:** Suppose you try 3 windows, 2 thresholds and 2 `skip_epochs` values for one signal, and keep whichever version looks best. The derivation counts the versions and estimates the chance that at least one reaches $\rho \geq 0.9$ by luck alone, treating the versions as independent. They usually aren't, so read the result as a rough picture, not an exact value.

**Derivation (H.K. Dass layout): the chance that at least one version reaches $\rho \geq 0.9$ by luck**

**Given:**

1. 3 windows, 2 thresholds, 2 `skip_epochs` values (illustrative).
2. By chance, one version reaches $\rho \geq 0.9$ with probability $\frac{1}{24}$ (Section 5.4).
3. The versions are treated as independent (simplifying assumption).

**Let:**

1. $V$: the number of versions tried. Integer. Role: how many chances there are.
2. $\Pr_{\text{none}}$: chance that no version reaches $\rho \geq 0.9$. Scalar in $[0, 1]$.
3. $\Pr_{\text{any}}$: chance that at least one version does. Scalar in $[0, 1]$.

**To find:** $V$ and $\Pr_{\text{any}}$.

**Solution:**

$$
\begin{aligned}
V &= 3 \times 2 \times 2 && \because \text{every combination of the settings in Given 1} \\
&= 12 && \because \text{multiplication} \\
\Pr_{\text{none}} &= \left(1 - \tfrac{1}{24}\right)^{12} && \because \text{each of the 12 versions misses with probability } 1 - \tfrac{1}{24} \text{ (Given 2, 3)} \\
&= \left(\tfrac{23}{24}\right)^{12} && \because 1 - \tfrac{1}{24} = \tfrac{23}{24} \\
\Pr_{\text{any}} &= 1 - \left(\tfrac{23}{24}\right)^{12} && \because \text{“at least one” is the opposite of “none”} \\
&\approx 1 - 0.600 && \because (23/24)^{12} = 0.6000\ldots, \text{ rounded to 3 d.p.} \\
&= 0.400 && \because \text{subtraction}
\end{aligned}
$$

$\therefore$ One version has about a 4% chance of reaching $\rho \geq 0.9$ by luck; picking the best of 12 raises that to about 40%. That's why settings must be fixed before you look.

$$\boxed{V = 12, \quad \Pr_{\text{any}} \approx 0.400}$$

**Self-check:** Which settings in your config say "not from a paper"? (Check each window and skip value in `nanda_unified.yaml` for a citation.)

**Likely viva question:** "How many rules did you try before this one?"

## 7.4 Sensitivity analysis

**Definition:** Sensitivity analysis (**N49**) is changing one setting a little and checking whether the verdict changes.

**Why it matters:** If a verdict flips when a threshold or window moves a little, it's fragile. Near-misses show where that can happen.

**Worked example:** Suppose a rule fires at $P_i = 0.09\,G_i$ in every seed (an illustrative list, **N32**). All five seeds lead and the order of the $G_i$ is kept, so lead_frac $= 1.0$ and $\rho = 1$, as in Section 6.3. The derivation measures how far it misses the upper limit on $\bar{r}$.

**Derivation (H.K. Dass layout): how far the list $P_i = 0.09\,G_i$ misses**

**Given:**

1. $P_i = 0.09\,G_i$ for every seed (illustrative list above).
2. $r_i = (G_i - P_i)/G_i$ and $\bar{r} = \frac{1}{n}\sum_{i \in F} r_i$ (**N37**), with $n = 5$.
3. Upper limit on $\bar{r}$: 0.9 (**N43**).

**Let:**

1. $r_i$: the relative lead of seed $i$. Scalar.
2. $\bar{r}$: the mean relative lead of this list. Scalar.
3. $\delta$: the miss, $\delta = \bar{r} - 0.9$. Scalar; positive means above the limit.

**To find:** $\delta$.

**Solution:**

$$
\begin{aligned}
r_i &= \frac{G_i - 0.09\,G_i}{G_i} && \because \text{ Given 2 with Given 1} \\
&= \frac{0.91\,G_i}{G_i} && \because \text{ subtraction} \\
&= 0.91 && \because \ G_i/G_i = 1 \\
\bar{r} &= \frac{5 \times 0.91}{5} && \because \text{ all five } r_i \text{ equal 0.91 (Given 2)} \\
&= 0.91 && \because \text{ division} \\
\delta &= \bar{r} - 0.9 && \because \text{ definition of } \delta \text{ (Let 3)} \\
&= 0.91 - 0.9 && \because \text{ substitution of the line above} \\
&= 0.01 && \because \text{ subtraction}
\end{aligned}
$$

$\therefore$ This list fails only by 0.01. An upper limit of 0.92 instead of 0.9 would turn it into a PASS. A verdict that hangs on the second decimal of a threshold needs a sensitivity

check.

$$\boxed{\delta = 0.01}$$

**Self-check:** Which threshold would you vary first for a near-miss like this? (The one it misses by the least: here the upper limit 0.9 on $\bar{r}$.)

**Likely viva question:** "Would your verdict survive a different window size?"

## 7.5 Baselines, controls and noise floor

**Definition:** A control run (**N50**) is one where grokking doesn't happen, or a setting is switched off. The noise floor is how much a signal wobbles for no reason.

**Why it matters:** Without them you can't say a change in a signal is because of grokking. A noise floor is only useful if a rule actually uses it to decide when an event is real.

**Worked example:** A run with weight decay $\lambda = 0$ (**N19**), which often doesn't grok, would show what any signal $y_t$ does without grokking. There's no calculation here.

**Self-check:** How would you estimate the noise floor from the saved curves?

**Likely viva question:** "Where's your control condition?"

## 7.6 Negative results and reproducibility

**Definition:** A negative result (**N51**) is a clear "this doesn't work". Reproducibility means someone else can rerun your work and get the same numbers.

**Why it matters:** Seven of your eight predictors are closed-negative (**N45**), so the thesis's value is in showing the tests were fair and repeatable: fixed seeds, saved data, frozen thresholds.

**Worked example:** Recomputing from the saved accuracy curves gives the same $G_i$, and rebuilding the split with the same seed reproduces the saved accuracies. There's no calculation here.

**Self-check:** What makes a negative result publishable rather than just a failure?

**Likely viva question:** "If almost nothing passes, what is the contribution of the thesis?"

# Recommended reading


[1] Power, A., Burda, Y., Edwards, H., Babuschkin, I., Misra, V. (2022). *Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets.* arXiv:2201.02177. (Modules 1–2.)

[2] Nanda, N., Chan, L., Lieberum, T., Smith, J., Steinhardt, J. (2023). *Progress Measures for Grokking via Mechanistic Interpretability.* ICLR 2023. (Modules 1–2; your setup.)

[3] Loshchilov, I., Hutter, F. (2019). *Decoupled Weight Decay Regularization.* ICLR 2019. (AdamW, Module 2.)

[4] Spearman, C. (1904), the original paper, or any introductory statistics chapter on Spearman rank correlation. (Module 5.)

[5] Nosek, B. A. et al. (2018). *The Preregistration Revolution.* PNAS 115(11). (Modules 6–7.)